

CS441 Assignment 2 Report

20220019 안재영

1. Assignment overview

본 과제는 HOG (Histogram of Oriented Gradients) 특징과 선형 SVM (Support Vector Machine) 분류기를 결합하여 이미지 내에서 보행자를 다중 스케일로 정확하게 탐지하는 것을 목표로 한다. HOG는 객체의 형태 정보를 추출하는 기능을, SVM은 이를 기반으로 한 이진 분류를 수행하며, 구현은 크게 4개 단계로 구성된다.

1. HOG 특징 추출: 이미지의 x, y 방향 기울기를 계산하여 구조 및 경계를 파악하고, 기울기의 크기와 방향을 셀 단위로 모아 방향 히스토그램을 생성한다. 이후 이중 선형 공간을 통해 정보의 견고성을 높이고, 블록 단위로 묶은 히스토그램에 L2-Hys 정규화를 실행해 조명 변화에 강인한 특징 벡터를 도출한다.
2. 선형 SVM 모델 학습: 객체의 존재 여부를 분류하는 선형 SVM 모델의 최적 가중치(w)와 편향(b)를 학습하는 과정. 모델의 학습에서 목적 함수로는 힌지 손실이 사용되며, 극값을 찾기 위해 확률적 경사 하강법을 통해 매개변수의 기울기를 반복적으로 계산하고 업데이트한다.
3. 다중 스케일 탐색: 고정된 크기의 이미지에 대한 분류를 수행하는 SVM 분류기의 유동성을 높이기 위해 원본 이미지를 여러 스케일로 리사이징하여 이미지 피라미드를 생성한다. 피라미드에 저장된 여러 스케일의 이미지마다 슬라이딩 윈도우 기법을 적용해 이미지의 모든 영역을 탐색하고, 각 윈도우에서 HOG 특징을 추출해 SVM 분류기로 전달한다.
4. 최종 분류: 각 탐지 후보 윈도우에 대해 SVM의 결정 함수 값인 신뢰도 점수를 계산하고, 그 중 임계값 조건을 만족하는 윈도우만을 남긴다. 또한 NMS를 적용해 동일한 객체를 대상으로 하는 겹치는 윈도우들 중 가장 높은 점수를 가진 박스만을 남긴다.

2. Implementation and Algorithm

TODO 1: Implement gradient computation

`compute_gradient` 함수의 목적은 이미지의 가로, 세로 방향 기울기 g_x, g_y 를 계산하는 것이다. 이 값은 이미지의 경계 및 특징을 추출하는 데에 사용된다. 기울기 값은 R, G, B 각 채널마다 별도로 계산하며, 이 때 중심 차분 방식을 사용한다. 이는 $[-0.5, 0, 0.5]$ 커널을 적용하는 것과 같으며, $g[i, j] = 0.5 * (I[i, j+1] - I[i, j-1])$ 의 식에 따라 계산된다. 다만 현재 이미지의 모서리 부분에는 중심 차분 계산에 대입하기 위한 이웃 픽셀이 존재하지 않으므로, `np.pad`를 사용해 이미지 가장자리에 픽셀을 패딩한다. 패딩 모드는 가장자리 패딩(edge padding)을 적용해 가장자리 픽셀을 복제하도록 한다.

HOG 특징 추출의 경우, 이미지에서 가장 확실한 경계를 사용하기 위해 세 채널의 기울기 값 중 가장 큰 것을 최종 결과로 선택한다. 따라서 제공된 계산으로 각각의 크기를 계산한 후, 최댓값을 찾아 반환한다.

TODO 2: Implement magnitude and angle computation

`compute_magnitude_angle` 함수의 목적은 앞서 계산한 가로, 세로 방향의 기울기를 이용해 각 픽셀에서 기울기의 크기와 그 벡터 방향을 계산하는 것이다. 이 값들은 HOG에서 특징을 구성하는 핵심 정보로 사용된다. 각각의 값은 제곱근과 아크탄젠트에 의해 계산된다. 이 때 방향의 경우 HOG가 부호 없는 방향을 사용함에 따라 $-180 \sim 180$ 범위가 아닌 $0 \sim 180$ 범위로 변환해야 하는데, `np.mod` 연산을 사용해 계산한다.

TODO 3: Implement cell histogram computation

`compute_cell_histogram` 함수는 HOG 특징의 핵심 단계를 구현한다. 그 목적은 앞서 계산한 이미지의 기울기 크기 및 방향을 사용해 각 셀별로 기울기 방향 히스토그램을 생성하는 것이다. 이를 통해 각 셀의 로컬한 이미지 경계 정보를 압축할 수 있다.

전체 이미지를 $H \times W$ 크기에서 가로, 세로 `cell_size`의 셀들로 분할한다. 그 후 $0 \sim 180$ 의 방향 범위를 `nbins`로 나누어 각 히스토그램의 빈 크기(`bin_size`)를 결정하고, 이 간격에 따라 연속적인 빈 인덱스(`bin_idx`)를 계산한다.

그 뒤에는 HOG의 핵심 원리 중 하나인 이중 선형 보간을 통해 기울기 크기를 인접한 빈에 분산시켜 할당한다. 이 특징은 히스토그램이 경계에서의 작은 노이즈나 변화에 받

는 영향을 줄여, 특징의 견고성을 높인다. 이를 위해 빈 인덱스에서 정수 부분을 추출하고, 그 lower bin에 1을 더한 값으로 upper bin까지 계산한다. 이 때 180과 0은 같은 방향을 간주되므로, nbins를 사용한 모듈러 연산으로 인덱스들이 순환적으로 연결되도록 한다. 가중치는 upper bin에 $\text{bin_idx} - \text{lower_bin}$ 이 할당되며, lower bin에는 1에서 upper bin의 가중치를 뺀 값이 할당된다. 실제로 셀에 누적되는 히스토그램 값은 이 가중치를 픽셀의 기울기 크기에 곱한 것이다. 각 셀의 영역을 cy, cx 이중 반복문을 통해 순회하면서 셀 내부의 모든 픽셀에 대해 히스토그램 누적을 수행한다.

TODO 4: Implement block normalization

`normalize_block` 함수의 목적은 셀 히스토그램을 블록 단위로 묶어서 L2 정규화를 수행한 후, 정규화된 블록들을 1차원 벡터로 연결해 HOG의 특징 디스크립터로 만드는 것이다. 정규화는 조명이나 그림자 등 대비를 변화시키는 영향에 대해 특징의 불변성을 높이는 효과를 가진다.

블록은 가로, 세로 `block_size` 크기의 정사각형으로 정의되며, 각 블록은 겹쳐진 상태로 이동하면서 계산된다. 먼저 계산된 $n_blocks_y \times n_blocks_x$ 개의 블록들을 이중 반복문을 통해 순서대로 순회하면서 정규화를 수행한다. 먼저 각 블록의 block_size^2 개 셀 히스토그램을 추출한 후, `flatten` 함수를 통해 하나의 긴 벡터로 연결한다. 그 후 HOG의 표준 정규화 방식인 L2-Hys를 적용해 특징을 더욱 견고하게 하는데, 이 방식은 1차로 벡터를 L2 norm으로 나눠 크기를 1로 만들고(이 때, 작은 ϵ^2 이라는 작은 값을 분모에 더해 수치적 안정성을 확보), 임계값 0.2를 기준으로 클리핑한 후, 다시 한번 L2 정규화를 수행해 정규화 상태를 복원한다.

이와 같은 과정으로 정규화한 후 각 블록 히스토그램 벡터를 순서대로 HOG의 특징 벡터 `features`에 연결해 반환한다.

TODO 5: Implement HOG feature extraction

`extract` 함수는 HOG의 디스크립터인 `HOGDescriptor` 클래스의 메인 인터페이스로, 입력된 이미지에 대해 HOG 특징 추출 파이프라인을 순서대로 실행하고 최종 벡터로 반환하는 기능을 수행한다.

과정은 총 네 가지로 구분할 수 있다. 그 순서는 각각 기울기 계산, 크기와 방향 계산, 셀 히스토그램 계산, 블록 정규화 및 특징 추출로, 각각 이미 해당 기능의 함수들을 구현하였으므로 적절하게 호출하는 것으로 수행 가능하다.

TODO 6: Implement hinge loss computation

`compute_hinge_loss` 함수의 목적은 선형 SVM 모델의 힌지 손실을 계산하는 것이다. 힌지 손실은 모델의 잘못된 분류에 페널티를 부과하며, SVM 학습은 이 손실을 최소화하는 방향으로 진행된다.

SVM 모델은 예측값을 $w^T x + b$ 로 계산하며, 마진은 실제 레이블($y = -1$ or 1)과 예측값의 곱으로 계산된다. 그 값이 1보다 큰 양수일 때, 샘플은 올바르게 분류되었으며 결정 경계로부터 충분히 떨어진, 손실이 없는 값을 의미한다. 반대로 마진이 1 미만일 경우에 힌지 손실의 페널티를 부과하며, 이는 수학적으로 $\max(0, 1 - \text{margin})$ 으로 표현할 수 있다. 이 값을 전체에서 평균을 냄으로써 데이터 손실을 계산한다.

최종 SVM의 목적 함수는 데이터 손실과 정규화 손실의 합으로 구성되며, 정규화 손실은 모델의 복잡도(w)의 크기를 제한함으로써 과적합을 방지하기 위한 목적이다. 이 때 정규화 매개변수 C 를 데이터 손실의 계수로 곱해 더함으로써, 이 파라미터의 조절을 통해 데이터 손실과 정규화 손실 간의 균형을 조절한다. 후술할 파라미터 조절에서 다루겠지만 C 가 클수록 데이터를 중시해 과적합 위험이 증가하며, 작을수록 w 의 크기가 작은 단순한 모델을 선호하게 된다.

TODO 7: Implement gradient computation

`compute_gradient` 함수의 목적은 선형 SVM의 힌지 손실 함수의 극값을 찾기 위해 사용되는 경사 하강법을 위해 가중치와 편향에 대한 손실 함수의 기울기를 계산하는 것이다. 앞서 구현한대로 힌지 손실 함수는 데이터, 정규화 손실로 구분되며, 기울기는 각각을 분리하여 계산 후 반환한다.

힌지 손실에서 데이터 손실은 마진 위반, 즉 마진이 1보다 작은 경우에 대해서만 적용되므로, 우선 마진과 1의 비교 결과를 `mask`에 저장해 기울기 계산에 사용하도록 한다. 손실 함수의 미분에 따라 가중치 w 에 대한 기울기 `grad_w`는 정규화 항의 기울기 w 에서 데이터 손실 항 기울기 $C * \text{mean}(y_i * x_i)$ 를 뺀 것이며, `grad_b`는 데이터 손실 항의 기울기만이 남은 $-C * \text{mean}(y)$ 로 계산된다. 이 때, `mean`의 계산에는 각 성분에 `mask` 인덱스를 고려하여 마진 위반을 일으킨 데이터만이 기여도를 가지도록 한다.

TODO 8: Implement SVM training with SGD

`train` 함수의 목적은 확률적 경사 하강법을 사용해 선형 SVM 모델의 최적 가중치 w 와 편향 b 를 도출하는 것이다. 주어진 `epoch`의 횟수동안 루프를 돌며 힌지 손실 함수를 최

소화하는 방향으로 매개변수를 갱신하는 알고리즘을 구현한다.

각 루프의 시작에는 훈련 데이터를 np.random을 사용해 무작위로 섞으며, 이는 데이터의 순서가 학습에 미치는 영향을 줄이고 local minimum 문제를 해결하기 위한 목적이다. 다음으로는 미니 배치에 대한 경사 하강법을 구현한다. 이 부분에서 전체 데이터셋은 batch_size 크기의 미니 배치로 나뉘어지고, 미니 배치 X_batch 및 y_batch와 현재의 w, b 값을 앞서 구현한 compute_gradient 함수에 전달해 그 반환값을 사용해 w, b를 새롭게 업데이트한다. 이 때 출력값에는 파라미터 learning_rate를 곱해 손실 함수의 최솟값을 찾아가는 경로의 속도와 정확도를 조절할 수 있도록 한다.

TODO 9: Implement image pyramid generation

generate_pyramid 함수의 목적은 다중 스케일 객체 감지를 위해 원본 이미지를 다양한 크기로 리사이징한 이미지 피라미드를 생성하는 것이다. 이를 통해 고정된 크기의 입력을 처리하는 분류기가 이미지 내에 존재할 수 있는 다양한 크기의 객체를 처리할 수 있도록 구현하며, 스케일 불변성을 간접적으로 높일 수 있다.

피라미드는 다운스케일링과 업스케일링을 분리하여 각각 진행하며, min_size와 max_scale로 제한되는 조건 내에서 scale_factor를 사용해 scale을 1부터 감소/증가시키면서 이미지를 resize_image 함수를 사용해 리사이징한다. 조정된 이미지는 pyramid에 추가되고, 모든 루프가 완료된 후 크기를 기준으로 sort 함수를 통해 정렬한다.

TODO 10: Implement sliding window

sliding_window 함수의 목적은 입력 이미지의 모든 위치를 탐색하는 슬라이딩 윈도우 기법을 구현하는 것이다. 이 알고리즘은 앞서 구현한 이미지 피라미드를 사용해 다양한 스케일과 위치에 존재하는 객체를 대상으로 분류기를 적용하는 기반을 만든다.

전달받은 window_size 변수와 image를 통해 윈도우와 이미지의 크기를 설정한 후, 이 변수를 사용해 이미지 범위를 이중 반복문으로 순회한다. 이 때 이동의 간격은 파라미터 step_size를 따르며, 이 값이 작을수록 정밀한 탐색이 가능하지만, 처리할 양이 많아지게 된다. 반복문에 따른 각각의 위치에서, 앞서 계산한 win_height x win_width 크기의 부분을 추출해 window에 저장한다. 추출된 윈도우는 해당 좌표 x, y와 함께 즉시 반환되는 대신, yield 키워드로 처리해 필요한 순간에 생성되도록 함으로써 메모리 사용량을 최소화하도록 구현되었다.

TODO 11: Implement Non-Maximum Suppression

`non_maximum_suppression` 함수의 목적은 오브젝트를 탐지하는 과정에서 생성된 겹치는 윈도우들 중 가장 유력한 박스만을 남기는 것이다. NMS는 모든 탐지 박스를 탐지의 신뢰도 점수 기준 내림차순으로 정렬하며, 신뢰도 점수는 `detections` 배열의 5번째 인덱스에 저장된 것을 읽어온다.

다음으로는 정렬된 배열을 대상으로 반복적 겹침 제거를 수행한다. 각 반복마다 가장 높은 점수를 가진 박스를 `best_det`로 선택하고, 최종 결과가 될 `kept_detections`에 추가한다. 해당 박스와 나머지 박스들의 겹침 정도를 `compute_iou` 함수를 사용해 계산하고, 그 값이 파라미터 `iou_threshold`보다 큰 경우 중복된 탐지로 판단하여 제거한다. 이 파라미터는 값이 작을수록 중복으로 판단하는 기준이 낮아져 더 많은 박스를 중복으로 판단해 제거하고, 탐지 결과가 더 엄격하게 억제되는 효과를 낳는다. 이와 같은 방식으로 박스 배열을 갱신하면서 더 이상 박스가 남지 않을 때까지 반복한 결과, 각 객체에 대해 가장 높은 점수를 가진 탐지 박스만이 최종 결과에 포함된다.

TODO 12: Implement Pedestrian Detection pipeline

`detect` 함수의 목적은 HOG 및 SVM 파이프라인의 요소들을 통합하여 단일 이미지에서의 객체 탐지를 완성하는 것이다.

가장 먼저 `generate_pyramid`를 호출하여 원본 이미지를 다양한 크기로 리사이징한 피라미드를 생성 및 저장한다. 그 후 이 피라미드에서 `scaled_image`와 그 `scale`을 순회하며 `sliding_window`를 적용해 다양한 크기의 이미지에 대해 윈도우를 그리고, HOG 디스크립터 벡터를 추출한 후 SVM 분류기의 예측에 대한 점수를 계산한다. 이 점수는 즉 해당 윈도우에 객체(보행자)가 존재하는 신뢰도를 의미한다. 그 후 이 점수가 `score_threshold` 파라미터보다 높은 값을 가질 경우에만 유효한 탐지 후보로 설정하여 해당 이미지의 `scale`을 기준으로 다시 원본 좌표 및 크기를 계산해 `all_detections`에 추가한다.

이렇게 피라미드를 순회하면서 탐지한 목록 `all_detections`는 마지막으로 겹치는 윈도우들 중 하나만을 남기는 NMS 과정을 거쳐 최종 반환한다.

3. Results (with hyperparameters)

SVM hyperparameters

모델에서 SVM 훈련에 사용되는 매개변수는 C , $learning_rate$, $epochs$, $batch_size$ 가 있고, 기본 수치는 각각 $C = 1.0$, $learning_rate = 0.001$, $epochs = 100$, $batch_size = 32$ 로 설정하였다. 이 때 Loss는 0.5827에서 최종 0.5730까지 점진적으로 감소하였으며, 각 계산 평균적으로 27it/s 정도의 속도로 수행되었다. 이를 바탕으로 한 SVM 테스트 정확도는 93.18%로 계산되었다.

정규화 매개변수 C 는 SVM의 목적 함수에서 데이터 손실과 정규화 손실의 합을 계산할 때 데이터 손실의 계수로 곱하는 값으로, 데이터 손실과 정규화 손실 간의 균형을 조절한다. C 가 클수록 데이터를 중시해 정확도는 높아지지만 과적합 위험이 증가하며, 작을수록 w 의 크기가 작은 단순한 모델을 선호하게 된다.

C 를 2.0으로 증가시킨 결과 SVM 테스트 정확도는 96.64로 증가하였지만, 훈련 데이터의 오분류 최소화해 비교적 치중하면서 Loss가 0.9426에서 최종 0.9798까지 점진적으로 증가하여 과적합이 의심되는 상황을 보여주었다. 반대로 0.5로 감소시킨 결과 Loss는 0.3397에서 0.3156으로 효과적으로 감소시켰지만, SVM의 테스트 결과는 62.43%로 데이터 학습의 신뢰도가 상당히 낮았다. 이는 정규화에 치중한 결과 데이터에 대한 과소적합이 발생한 것으로 보인다.

학습률 $learning_rate$ 는 경사 하강법에서 가중치(w)와 편향(b)을 반복적으로 갱신할 때 한번의 갱신에서 이동하는 크기를 결정한다. 값이 너무 클 경우 발산할 수 있으며, 너무 작을 경우 수렴하기까지 과도하게 많은 반복이 필요할 수 있다.

값을 0.01로 10배 증가시킨 결과 Loss는 일관되게 감소하지 않고 진동하는 모습을 보였다. 값을 0.00001로 줄인 결과에는 1.1394- $\rightarrow$ 0.8057로 일관되게 Loss를 감소시켰으나 SVM 테스트 결과는 48.53%로 크게 감소하였는데, 이는 epoch의 수가 최적점에 도달하기에 충분하지 않았기 때문으로 예상된다.

epoch는 전체 훈련 데이터셋을 순회하는 횟수로, 더 많은 반복은 모델이 올바른 최적점에 수렴하는 데에 도움이 될 수 있지만 마찬가지로 작동 시간이 증가한다. 이 변수의 테스트는 앞선 추측을 확인하기 위해 $learning_rate$ 0.00001의 환경에서 epoch를 1000으로

늘이는 방식으로 진행하였고, 그 결과 학습의 시간은 증가하였지만 그동안 Loss는 계속 감소하여 0.5827에 도달하였으며 SVM 테스트 결과는 76.29%로 상당히 높아졌다.

마지막 batch_size는 미니 배치의 크기로, 가중치를 한번 업데이트할 때 사용되는 데이터 샘플의 수를 결정한다. 그 수가 많을수록 확률적 경사 하강법의 속도는 감소하지만 기울기 계산에 노이즈가 줄어들어 업데이트의 안정성이 높아진다.

값을 10으로 감소시킨 결과 평균 약 33it/s로 학습은 매우 빨라졌지만 SVM 테스트 결과는 94.86%로 소폭 상승하였는데, 이는 작은 배치의 랜덤성이 local minimum의 탈출을 도와 일반화 능력을 향상시켰으며, 이 경우 일반화의 기여도가 안정성에 비해 유리하게 작용하였기 때문으로 예상된다.

window detection parameters

window detection에 사용되는 매개변수는 scale_factor, step_size, score_threshold, iou_threshold가 있고, 기본 수치는 각각 scale_factor = 1.2, step_size = 8, score_threshold = 0.2, iou_threshold = 0.1로 설정하였다.

scale_factor는 이미지 피라미드의 스케일 계수로 반복적인 다운스케일, 업스케일링으로 이미지 피라미드를 생성할 때 스케일을 줄이고 늘이는 비율이다. 값이 1에 가까울수록 스케일 간의 간격이 좁아 더 많은 이미지를 생성하고, 다양한 크기의 오브젝트를 검출할 확률을 높이지만 계산 부담이 증가한다.

step_size는 슬라이딩 윈도우의 이동 간격으로, 윈도우가 각 방향으로 이동하는 픽셀 간격이다. 숫자가 작을수록 세세한 검출 시도를 통해 높은 위치 정확도를 보장하지만, 마찬가지로 계산 부담이 증가한다.

score_threshold는 탐지 신뢰도의 임계값으로, SVM 분류기가 반환한 결정 함수 score을 유효한 탐지 박스로 인정하는 기준이다. 값이 높을수록 False Positive의 개수를 줄일 수 있다.

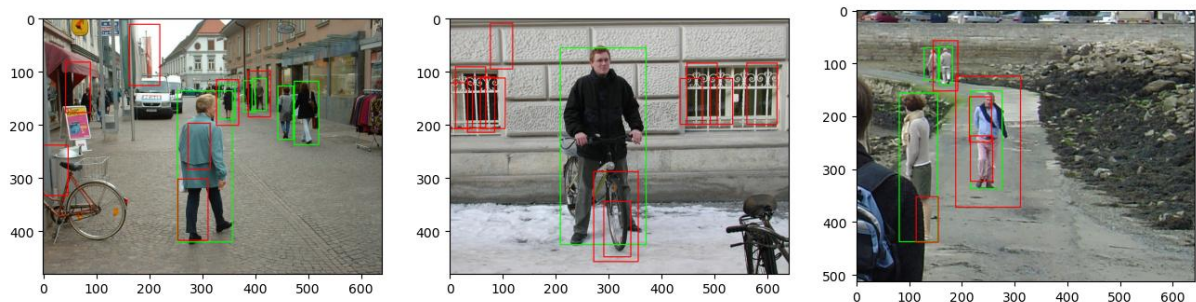
마지막 iou_threshold는 NMS의 IoU 임계값으로, 두 탐지 박스 간의 겹침 정도에 따라 박스를 제거하는 기준이다. 값이 작을수록 그 기준이 엄격한 것으로, 겹친 박스들을 더 적극적으로 제거하여 깔끔한 최종 결과를 얻을 수 있다.

다음은 scale_factor / step_size / score_threshold / iou_threshold에 따른 결과들이다.

1.2 / 8 / 0.2 / 0.1: 05:08 소요, 30.89s/it



1.1 / 4 / 0.6 / 0.4: 52:42 / 53:02 / 46:18 소요, 175.68 / 176.80 / 146.26 s/it



scale_factor와 step_size가 감소하면서 알고리즘의 수행 시간이 10배 가까이 증가하는 것을 확인할 수 있고, score_threshold의 증가로 인해 false positive가 어느정도 감소하고 실제 보행자를 나타내는 초록색 박스에 부합하는 정도가 비교적 높아진 것을 확인할 수 있다. iou_threshold의 증가는 또한 박스의 겹침 정도를 더 느슨하게 허용하여, 비교적 많이 겹쳐 있는 박스들을 확인할 수 있다.

Performance of sliding window detector

하지만 슬라이딩 윈도우 검출기의 결과는 상당히 실망스러웠는데, 실제 보행자를 나타내는 초록색 박스와 일치하는 true positive는 거의 발견되지 않았다. SVM의 90%가 넘는 정확도에도 불구하고 이러한 결과가 나오게 된 것은 슬라이딩 윈도우 자체의 불완전성에 의한 것으로 추정된다. 실제 이미지에서 슬라이딩 윈도우에 의해 생성된 윈도우들은 대부분이 보행자를 완전히 감싸지 못하고 부분만 포함하거나 배경을 많이 포함하고 있고, 따라서 고정된 크기의 훈련 데이터에 대해 측정된 SVM 분류기의 성능이 보장되지 않을 수 있는 것이다.

탐지 성능을 개선하기 위해서는 모델의 성능을 향상시키거나 탐색의 효율을 높이는 두

가지 방향을 고려할 수 있다. 먼저 훈련 단계에서 SVM에 가장 많은 혼동을 주는 배경 윈도우를 찾아 훈련셋에 추가하고 모델을 재학습시키는 하드 네거티브 마이닝은 False Positive의 주 원인을 효과적으로 구별하는 데에 도움이 될 수 있으며, HOG 이외에 LBP(Local Binary Patterns) 등의 다른 특징들을 추가하여 분류에 사용하는 특징을 풍부하게 만든다면 객체를 구별해내는 능력을 더 강화할 수 있다. 탐색의 효율성 증대를 위해서는 객체가 존재할 가능성이 높은 영역을 미리 결정한 후 해당 영역에 대해서만 슬라이딩 윈도우를 수행하는 선택적 탐색 방법을 고려할 수 있다.